

Deployment and Handover Guide

VPS acceptance, internal Linux migration and operational verification.

Edition	V0.1 / Delivery Package V1.0
Status	Implementation baseline
Date	2026-08-21
Target	Linux server
Runtime	Node.js, FFmpeg and Praat CLI
Service	PM2 or equivalent

Automatic outputs are research drafts. Researcher-reviewed TextGrid, transcript and matrix files are the final research data.

1. Handover model

Development and acceptance run on the Provider-managed VPS. After acceptance, the accepted release is migrated to the Client's internal Linux server. The handover package is generated from an accepted Git tag or commit, not from an untracked developer workspace.

2. Package contents

Item	Contents	Control
Source code	Versioned application, processing and Praat scripts	Accepted release ID
Deployment scripts	Install, build, start/stop, health and backup/restore	Tested on target server
Configuration templates	Environment variable names and non-secret defaults	No credentials
Documentation	Architecture, user guide and handover guide	Matches accepted release
Verification	Health, representative job and artifact download evidence	Recorded in acceptance checklist

3. Target environment

- Linux host with sufficient CPU, memory and storage for the agreed task size.
- Supported Node.js runtime and package-lock based install.
- FFmpeg/ffprobe and Praat CLI available on PATH.
- PM2 or an agreed service manager for background operation.
- TLS reverse proxy and authenticated network access where required by the institution.

4. Provisional deployment sequence

Step	Action	Verification
1	Install runtime dependencies and copy the accepted source archive.	Versions recorded
2	Run the locked package install and production build.	Build exits successfully
3	Create server-side environment configuration with approved credentials.	No key appears in source/log output
4	Start the service under PM2 or equivalent.	Process remains online
5	Check health and open the Direct Web Interface.	HTTP and browser checks pass
6	Run one authorized representative task and download artifacts.	Timeline and download checks pass
7	Record backup/restore and deletion controls.	Operational checklist signed

5. Configuration and secrets

- Use the supplied ``.env.example`` only as a key-name template.
- Enter AssemblyAI and pyannoteAI credentials directly on the server.
- Restrict file permissions and do not place credentials in Git, screenshots or delivery logs.
- Provider/model changes require the approved configuration and validation process.

6. Data operations

Control	Requirement
Storage	Keep runs outside the source tree and separate source audio, working files and accepted artifacts.
Backup	Back up only according to the Client's institutional policy.
Deletion	Delete acceptance-VPS research data using the agreed destruction protocol after migration/acceptance.
Evidence	Record deletion actions and provider job references when available.

7. Operational verification

Check	Pass condition
Build	Locked dependency install and production build succeed
Service	Process remains online and health endpoint responds
WebUI	Layer navigation and release information render correctly
Processing	Representative authorized job completes or reaches its expected human gate
Artifacts	Authorized download works and path traversal is rejected
Secrets	No credential is present in source, build output or public logs
Recovery	Restart and agreed backup/restore procedure are demonstrated

Acceptance boundary

The 90-day warranty covers defects against the accepted specification. Method, architecture, schema or deliverable changes remain separate change decisions.

8. Handover completion

Handover completes when source and deployment materials are delivered, the internal-server verification checks pass, open items are recorded, and both parties identify the accepted release in the Acceptance Checklist.